Multilayer Perceptron
多层感知机（MLP）

目录/CONTENTS

01 感知机

02 单层感知机局限性

03 多层感知机

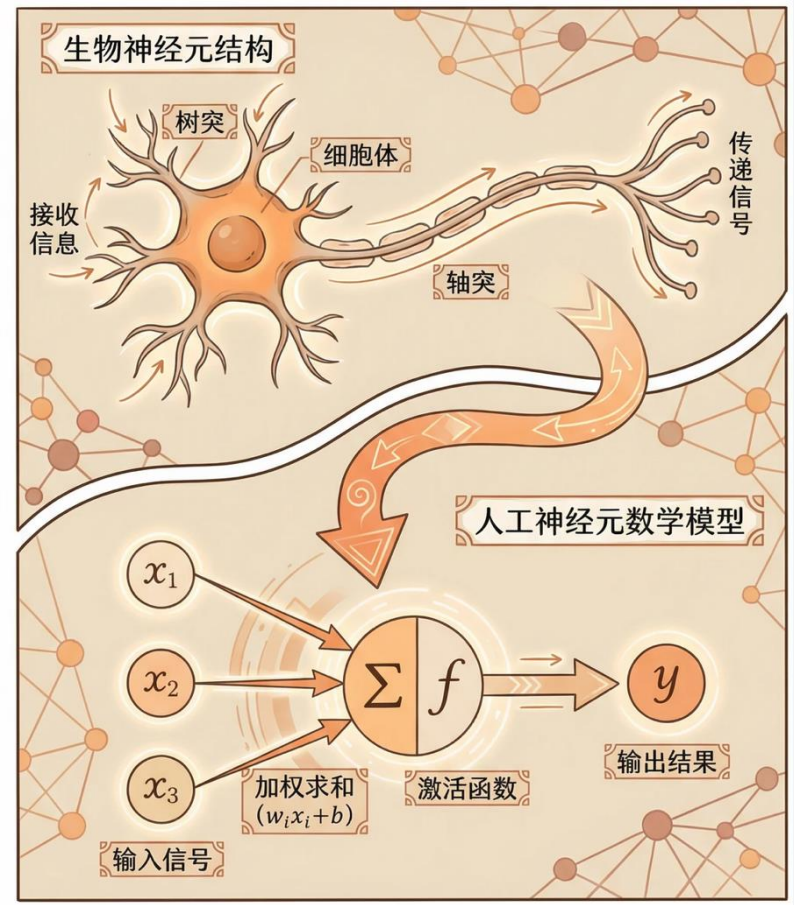
04 多层感知机的基本原理

05 多层感知机的总结

01

感知机

起源



起源：

1957年，美国科学家Frank Rosenblatt提出了感知机，这是最早的人工神经络模型之一。它的灵感来源于生物神经元：接收输入、加工处理、产生输出。

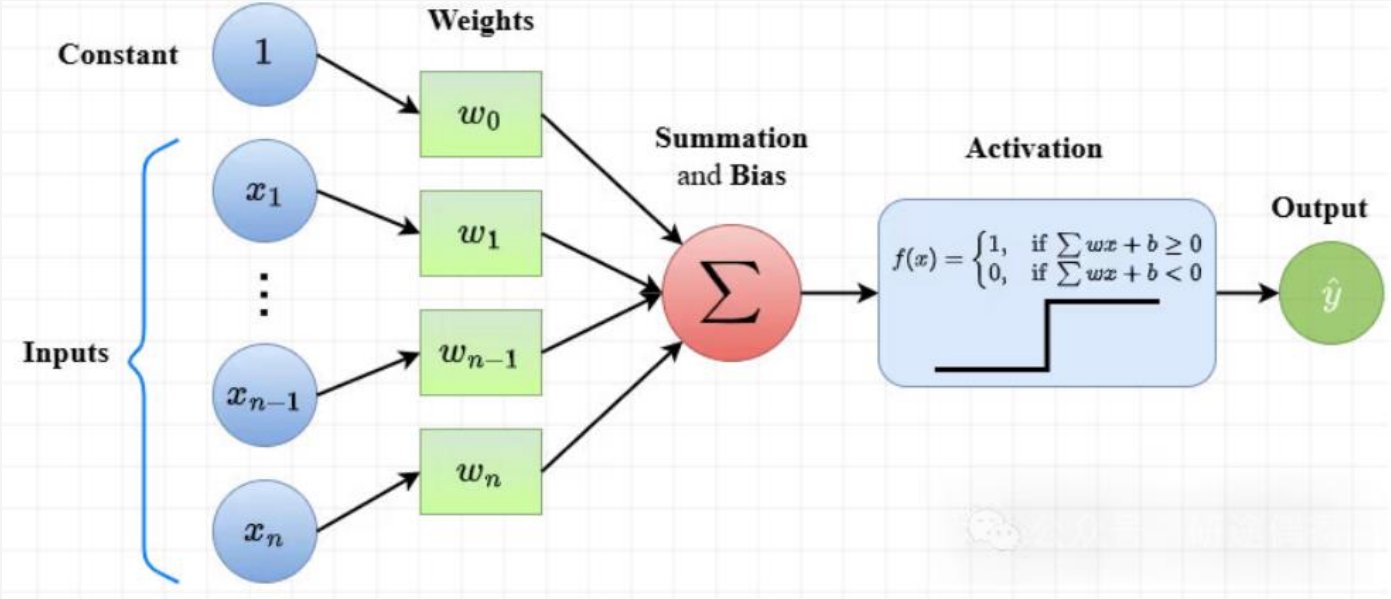
感知机（perceptron）简单介绍：

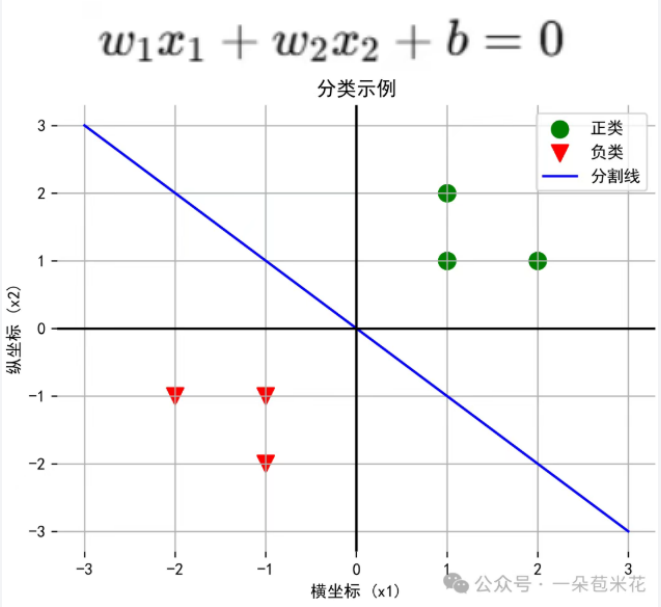
- 只有输入层和输出层，是**二分类的线性分类模型**，属于监督学习算法。
- 输入为实例的**特征向量**，输出为实例的**类别（取+1和-1）**。
- 旨在求出将输入空间中的实例划分为**两类的分离超平面**。
- 为求得超平面，感知机导入了**基于误分类的损失函数**，利用**梯度下降法**对损失函数进行最优化求解。

工作原理

单层感知机通过加权求和输入信号并加上偏置值，然后经过阶跃激活函数处理，输出二分类结果。

- **加权求和**：输入信号被送往输出层时，会被分别乘以各自的权重，然后求和。
- **偏置值**：用于**调整**输出层的**激活阈值**。
- **激活函数**：在单层感知机中，常用的**激活函数是阶跃函数**，它将大于某个阈值的结果输出为1，小于阈值的结果输出为0。





02

单层感知机局限性

局限性

感知机的“致命缺陷”：线性可分的局限

1969年，马文·明斯基（Marvin Minsky）在《Perceptrons》中指出：单层感知机**无法解决非线性可分**问题——最经典的例子是**异或（XOR）问题**：

异或的决策边界是**非线性的**，而单层感知机**只能处理线性可分问题**——直接导致了人工智能的**第一次“寒冬”**

03

多层感知机

引入

多层感知机的诞生：用“堆叠”突破线性局限

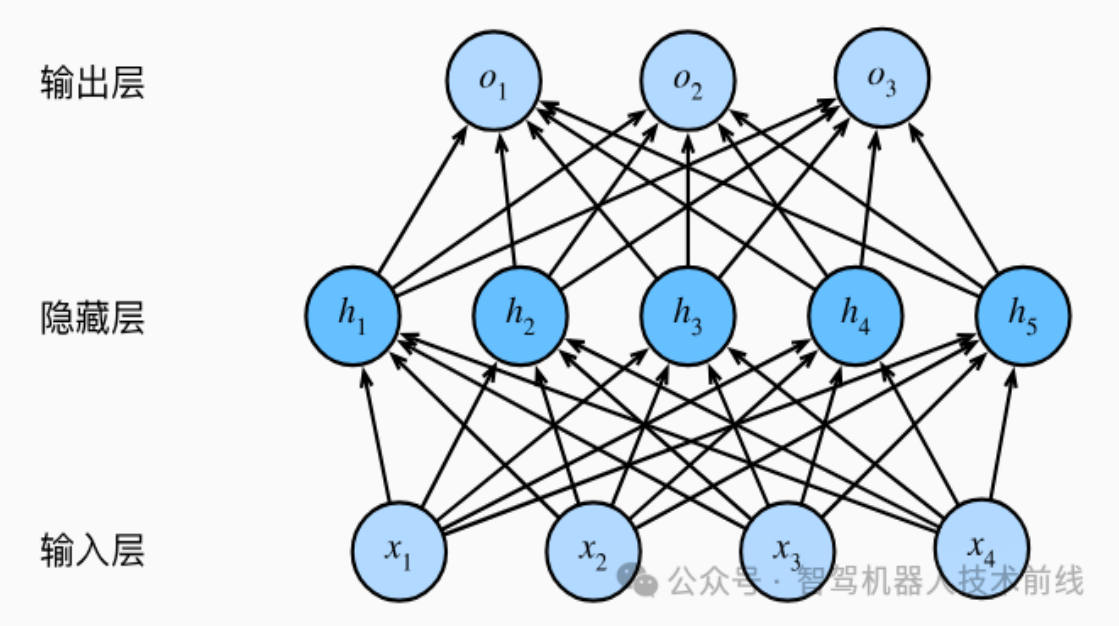
- 直到1986年，杰弗里·辛顿（Geoffrey Hinton）等人提出反向传播算法（BP），解决了“多层模型如何训练”的问题，MLP（多层感知机）正式诞生。从此，MLP成为了**深度学习的基础**。
- MLP的核心改进是**增加隐藏层**：在输入层和输出层之间**插入1层或多层神经元**，用于提取输入的**高阶特征**。

什么是多层感知机?

多层感知机是一种简单但强大的人工神经网络模型。由**多层“神经元”组成**，常包括：

- **输入层**：接收外部数据，MLP的**“入口”**，它的神经元数量等于输入数据的特征数。
- **隐藏层**：处理数据，MLP的**“加工厂”**,每层的神经元数量是超参数（需要自己调）。隐藏层越多、神经元越多，模型越复杂，但也**更容易过拟合**。
- **输出层**：给出最终预测结果。MLP的**“出口”**。

MLP属于**前馈神经网络。从数据中学习规律，然后用来做预测**，比如判断图片里是猫还是狗，或者预测房价。



上图是**单隐藏层的MPL**。输入层不涉及任何计算，故产生输出只需要实现隐藏层和输出层的计算，**层数为2**。

多层感知机

核心思想：用“非线性堆叠”模拟任意复杂函数

核心思想：

MLP的本质是**多层线性变换+非线性激活的组合**，总结如下：

1.多层堆叠：通过“输入层→隐藏层→输出层”逐层提取抽象特征；

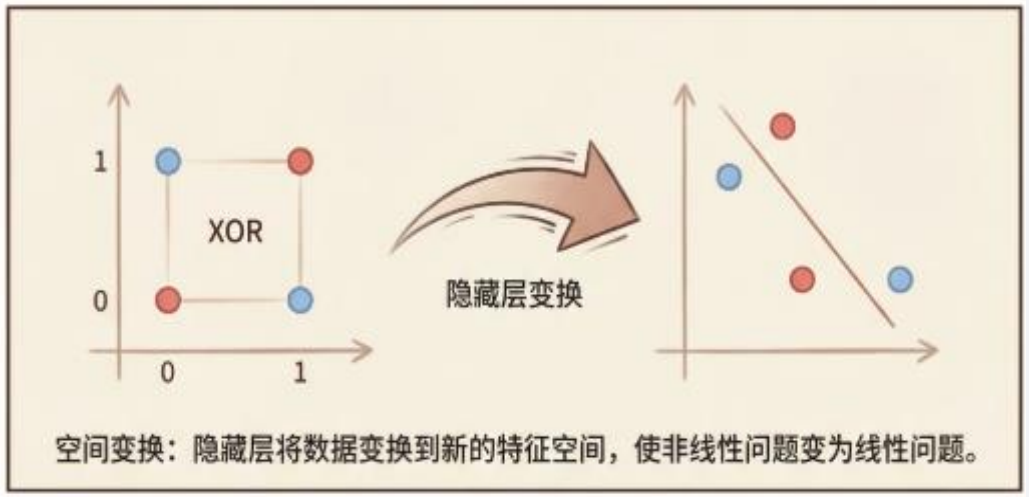
2.非线性激活：每一层输出必须经过非线性函数处理，否则多层结构会退化为单层线性模型（后续将证明）。

例如，解决异或问题的MLP结构：

输入层：2个神经元（对应 x_1, x_2）

隐藏层：2个神经元（提取特征，如 $h_1 = \sigma(x_1 + x_2 - 0.5)$ ）

输出层：1个神经元（用隐藏层特征做线性组合）



空间变换：隐藏层将数据变换到新的特征空间，使非线性问题变为线性问题。

多层感知机

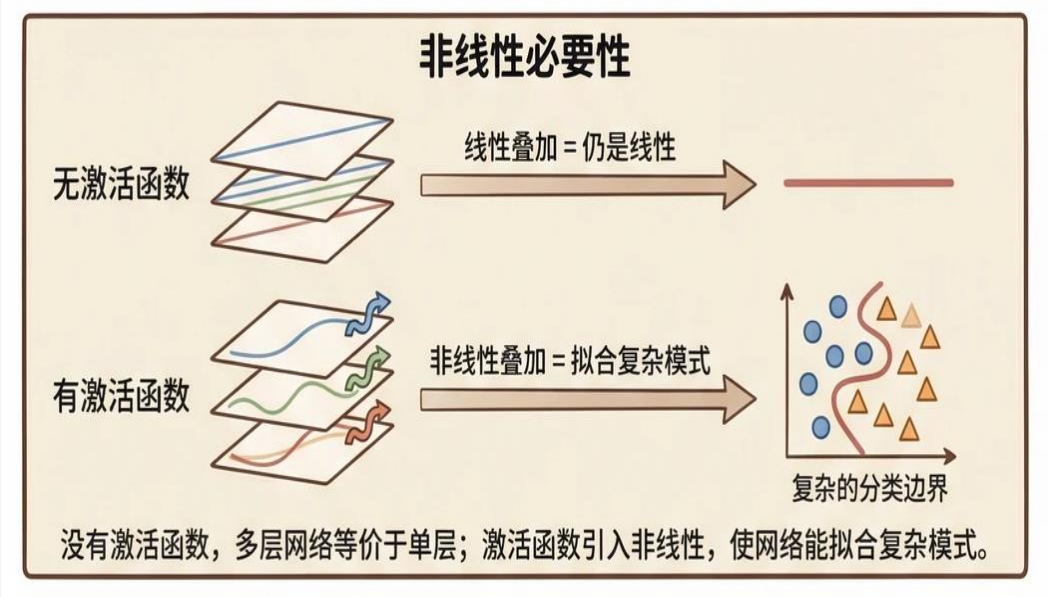
非线性激活必要性

为什么需要“非线性激活”？

假设我们有一个两层无激活的模型（输入层→隐藏层→输出层，均不用激活函数）：

- 隐藏层输出：$\mathbf{h} = W_1\mathbf{x} + b_1$
- 输出层输出：$\hat{y} = W_2\mathbf{h} + b_2 = (W_2W_1)\mathbf{x} + (W_2b_1 + b_2)$。

会发现：**两层无激活的模型等价于单层线性模型**（权重是W_1W_2，偏置是$W_1b_1 + b_2$）也就是说，**没有非线性激活，多层结构毫无意义**——这是MLP的“灵魂定理”。



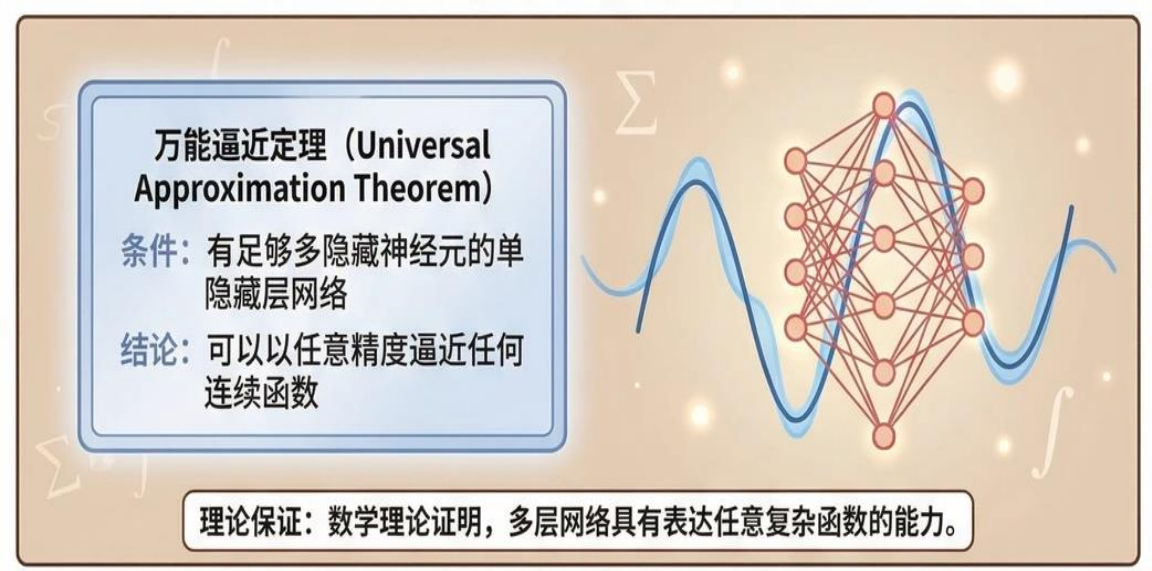
多层感知机

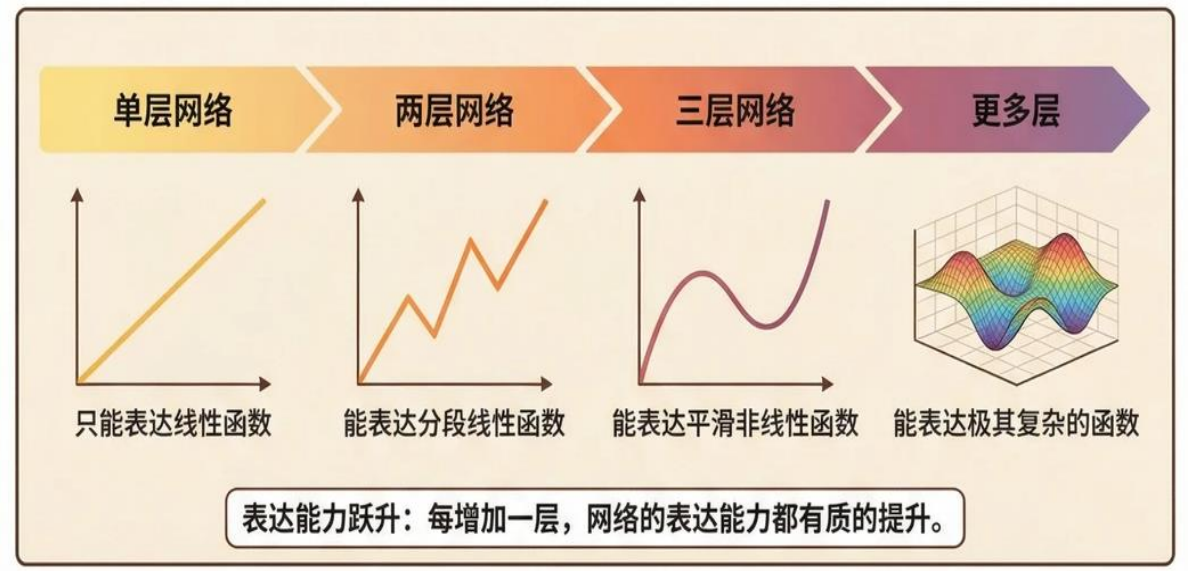
万能近似定理的数学保证

万能近似定理：MLP能拟合任意连续函数

1989年，霍赫赖特（Hornik）等人证明了**万能近似定理**：只要隐藏层神经元足够多，MLP可以以任意精度逼近定义在紧集上的连续函数。

意味着：MLP理论上能解决所有“连续输出”的问题——从房价预测到图像分类，只要设计合理的结构。





04

多层感知机的基本原理

MLP的基本原理

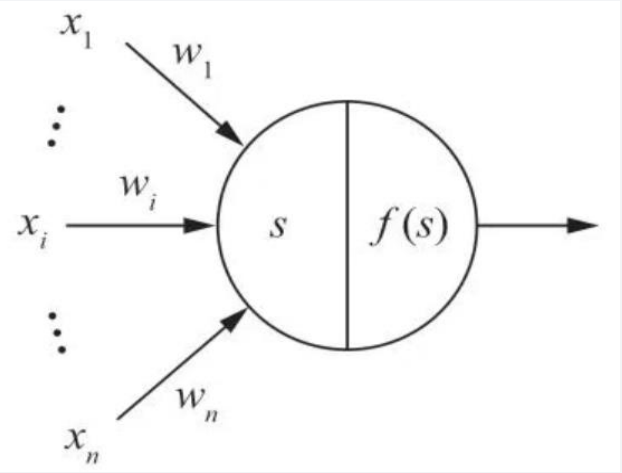
单层神经元的计算逻辑

第 l 层到 l+1层的计算分为两步：

1. 线性变换：
上一层激活**输出加权求和+偏置**，得到预激活值

2.非线性激活
预激活值**经过激活函数**，得到当前层输出。

单个神经元工作原理：



数学公式：$z^{(l+1)} = W^{(l)}a^{(l)} + b^{(l)}a^{(l+1)} = \sigma(z^{(l+1)})$ 其中 $\sigma(\cdot)$ 是激活函数，是MLP的“核心开关”

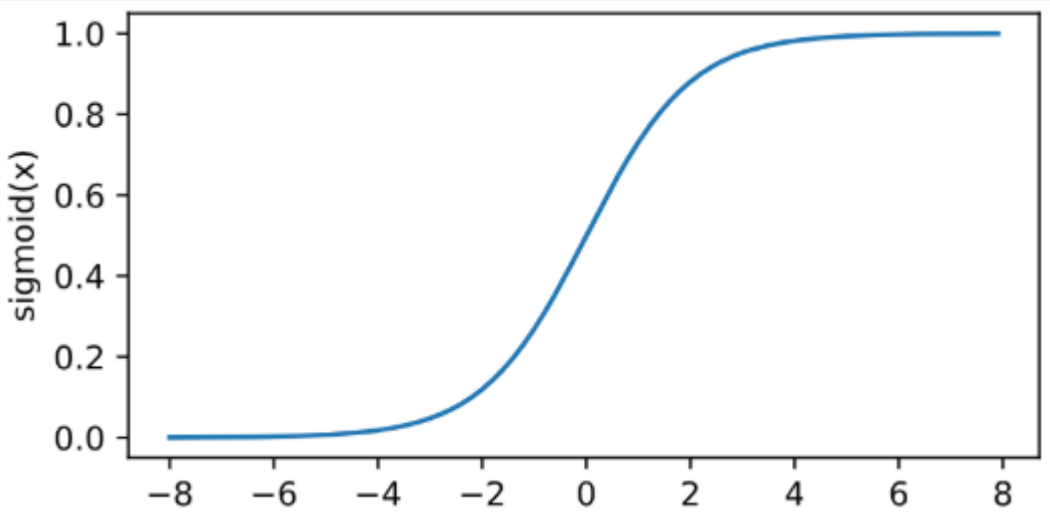
MLP的基本原理

常见的激活函数

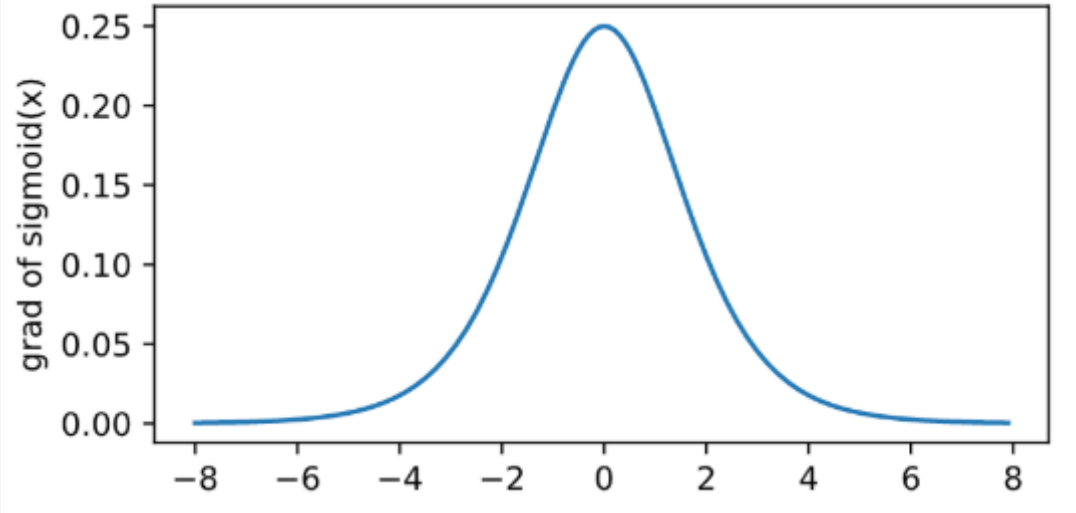
1. Sigmoid函数：二分类的“概率开关”：

$$f(x) = \frac{1}{1+e^{-x}}$$

输出范围在 (0,1) 区间，常用于**二分类输出层**。



导数（反向传播需要）：$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$



优点：输出有概率解释；

缺点：易出现“梯度消失”（当z很大/很小时，导数趋近于0，导致梯度无法传递）；

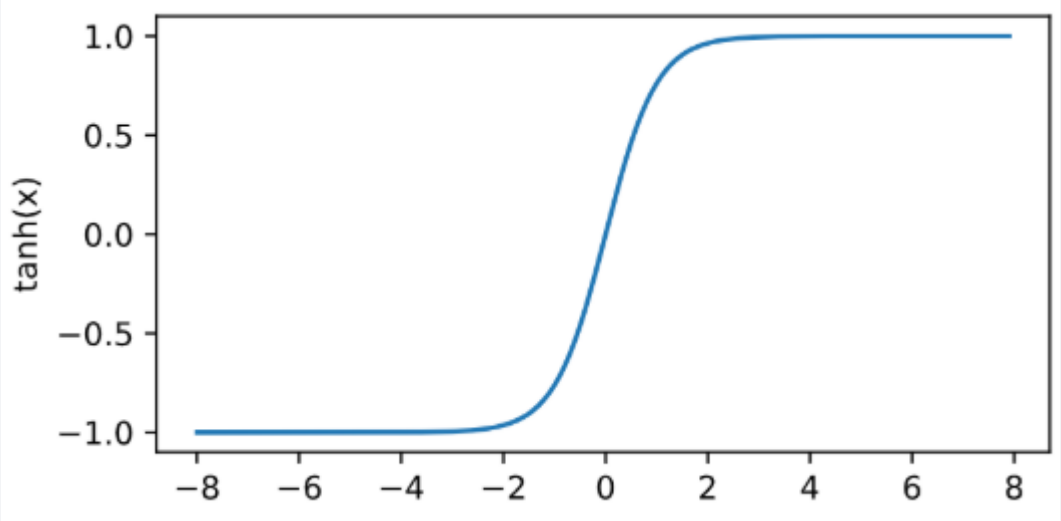
使用场景：二分类、概率输出

MLP的基本原理

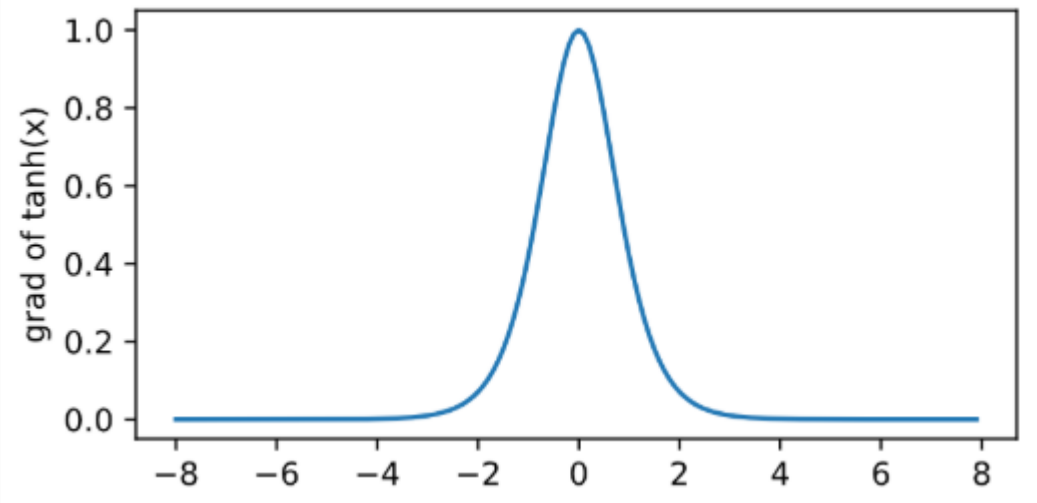
2.Tanh函数（双曲正切函数）：

$$f(x) = \tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

输出范围在 (-1,1) 区间，**关于坐标原点中心对称**。



导数：$\tanh'(x) = 1 - \tanh^2(x).$



优点：零中心化，收敛比 Sigmoid 快；

缺点：同样有梯度消失问题;

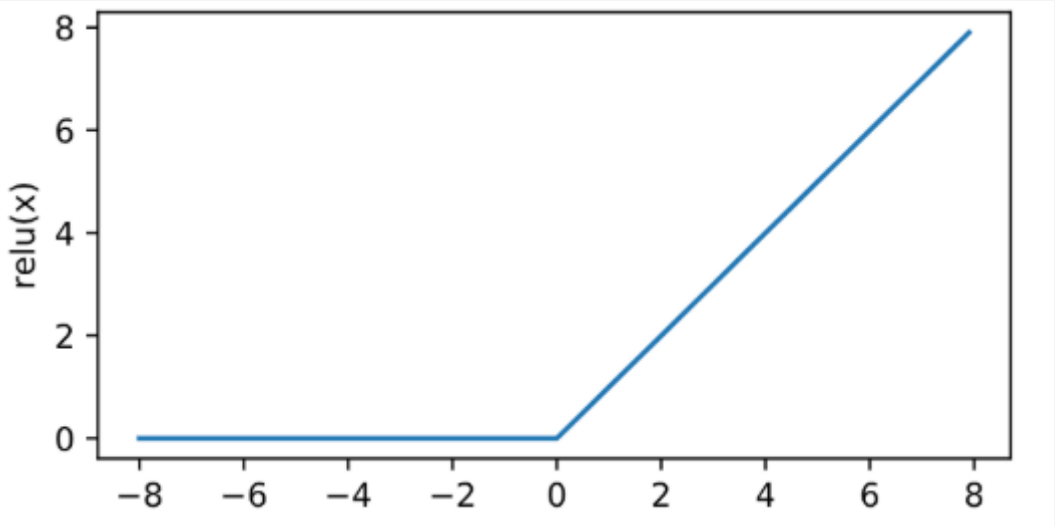
使用场景：RNN、某些隐藏层

MLP的基本原理

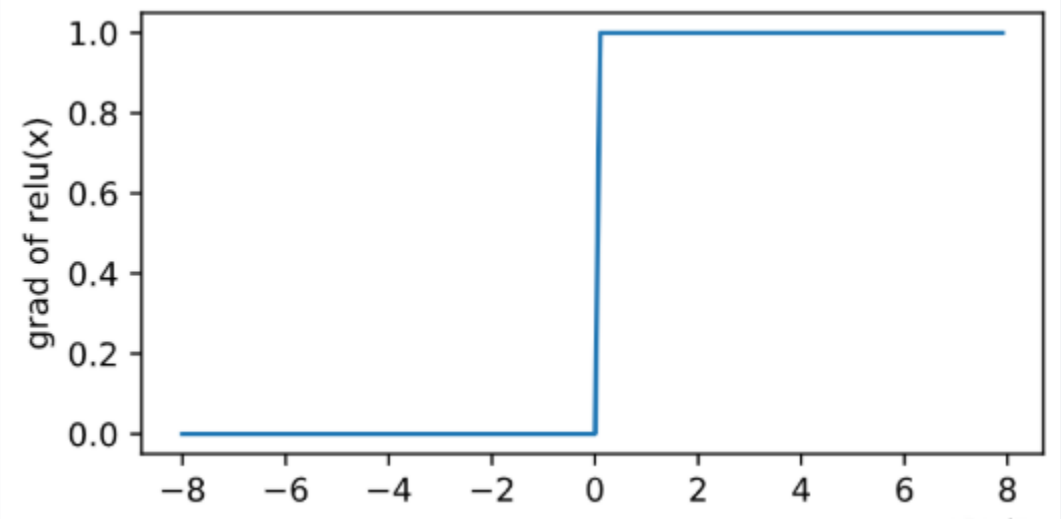
3.ReLU函数：当代深度学习的“主流选择”

$$\mathrm{ReLU}(z) = \max(0, z)$$

小于0时输出0，大于0时保持原值，**简单又高效。**
修正线性单元，解决了梯度消失问题，是**隐藏层的默认激活函数。**



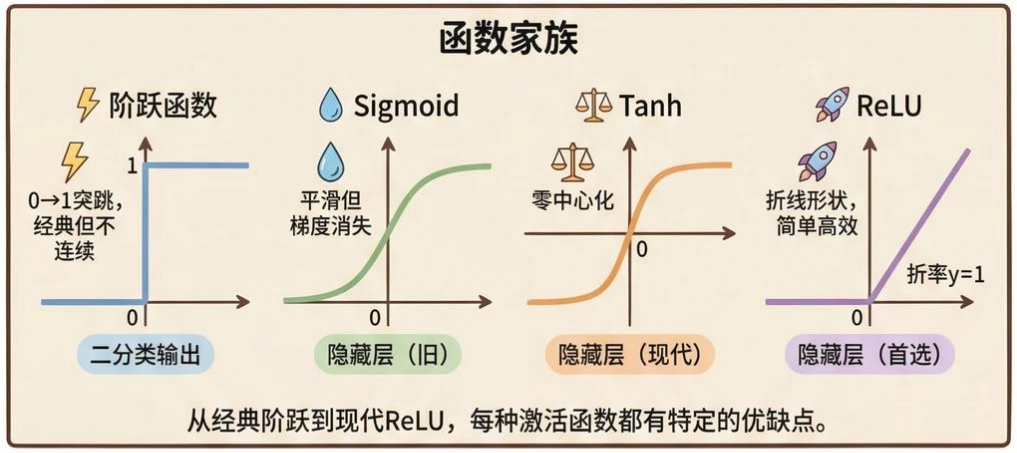
导数：$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$

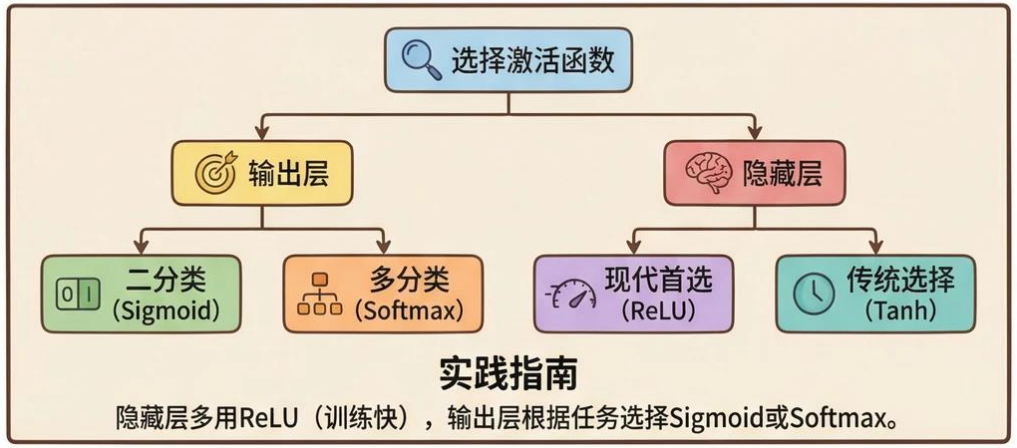


优点：计算快；避免梯度消失（z>1时导数为1）；
缺点：存在“死亡ReLU”(z≤ 0时神经元无法更新，可改用Leaky ReLU $\max(0.01z, z)$
使用场景：绝大多数现代神经网络隐藏层

MLP的基本原理

常见的激活函数总结与选择





输出层的激活函数选择：

输出层激活函数需根据**任务类型**决定：

- **回归问题**（预测连续值，如房价）：
  用恒等函数（$\sigma(z) = z$，直接输出线性结果）；
- **二分类问题**（如垃圾邮件）：
  用Sigmoid（输出0~1的概率）；
- **多分类问题**（如手写数字识别）：
  用Softmax（$\mathrm{Softmax}(z_i) = \frac{e^{z_i}}{\sum e^{z_j}}$）
  （将输出转化为概率分布，满足 $\sum_{i=1}^{k} \hat{y} * i = 1$）。

MLP的基本原理

模型训练——“前向→损失→反向→更新”的循环

（1）前向传播：

前向传播是MLP的**“计算过程”**，数据从输入层流向输出层。从输入到输出，整个计算流程是：

输入数据：

$$\mathbf{a}^{(0)} = \mathbf{x}$$

依次通过每一层：

$$\forall l = 1, \ldots, L:$$
$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$
$$\mathbf{a}^{(l)} = \sigma^{(l)}(\mathbf{z}^{(l)})$$

最后得到输出 $\mathbf{a}^{(L)}$，就是网络的最终预测结果。

（2）损失函数：

训练的目标是让模型预测ŷ尽可能接近真实值(y)。损失函数用来**衡量预测和真实的差距**，常见的有：

1. 回归问题：均方误差（MSE）

$$L = \frac{1}{2N}\sum_{i=1}^{N}(\hat{y}^{(i)} - y^{(i)})^2$$

2. 分类问题：交叉熵损失

- 二分类：二元交叉熵

$$L = -\frac{1}{N}\sum_{i=1}^{N}[y^{(i)}\log\hat{y}^{(i)} + (1-y^{(i)})\log(1-\hat{y}^{(i)})]$$

- 多分类：类别交叉熵

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{k} y_j^{(i)}\log\hat{y}_j^{(i)}$$

MLP的基本原理

（3）反向传播算法：

反向传播（Backpropagation）是**MLP训练的核心**。它利用**“链式法则”**,从输出层到输入层计算每个参数的梯度。核心是定义误差项：

$$\delta^{(l)} = abla_{z^{(l)}} L \text{（第 } l \text{ 层预激活值对损失的梯度）}$$

反向传播的步骤（以回归为例，32个样本）：

1. **输出层误差**：回归问题中，输出层用恒等函数，误差项为：$\delta^{(3)} = \hat{y} - y$
2. **隐藏层误差**：利用输出层误差反向计算：$\delta^{(2)} = \delta^{(3)} \cdot (W^{(2)})^T \odot \mathrm{ReLU}'(z^{(2)})$（$\odot$ 是元素-wise乘积，$(W^{(2)})^T$ 是权重矩阵的转置）；
3. **计算参数梯度**：
   - 隐藏→输出层权重梯度：$abla_{W^{(2)}} L = \frac{1}{32} a^{(2)T} \delta^{(3)}$；
   - 隐藏→输出层偏置梯度：$abla_{b^{(2)}} L = \frac{1}{32} \mathrm{sum}(\delta^{(3)}, \mathrm{axis} = 0)$；
   - 输入→隐藏层权重梯度：$abla_{W^{(1)}} L = \frac{1}{32} a^{(1)T} \delta^{(2)}$；
   - 输入→隐藏层偏置梯度：$abla_{b^{(1)}} L = \frac{1}{32} \mathrm{sum}(\delta^{(2)}, \mathrm{axis} = 0)$。

（4）参数更新：用优化器修正错误

优化器的作用是**根据损失的梯度更新参数，最小化损失**。

常见优化器：

1.梯度下降法（GD）

最基础的优化器，计算所有样本的梯度均值更新参数：

$$W^{(l)} \leftarrow W^{(l)} - \eta \cdot abla_{W^{(l)}} L \quad b^{(l)} \leftarrow b^{(l)} - \eta \cdot abla_{b^{(l)}} L$$

（η 是学习率，通常取0.001~0.1）

2. 随机梯度下降（SGD）

优化每次随机选1个样本计算梯度，更新参数：

$$W^{(l)} \leftarrow W^{(l)} - \eta \cdot abla_{W^{(l)}} L^{(i)}$$

（$L^{(i)}$是第 i个样本的

3. Adam优化器：当代“默认选择”

结合“动量”和“自适应学习率”，解决了SGD的波动问题，是MLP的最优选择：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 W^{(l)} \leftarrow W^{(l)} - \eta \cdot \frac{m_t}{\sqrt{v_t} + \epsilon}$$

m_t:动量；v_t:自适应学习率；$\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 1e-8$

05

多层感知机的总结

MLP的总结

适用场景

结构化数据：如表格数据（房价、客户 churn 预测）；
回归问题：预测连续值（房价、股票价格）；
简单分类问题：如垃圾邮件分类;
数据量适中：不需要海量数据（但至少需几百个样本）。

局限性与解决方法

1.易过拟合：

- 解决：正则化（L1/L2）、Dropout（训练时随机关闭神经元）、早停（验证集损失上升时停止）；

2.对特征缩放敏感：

- 解决：必须标准化/归一化；

3.参数多，调参麻烦：

- 解决：从简单模型开始（如1层隐藏层），用自动调参工具（如Grid Search）。

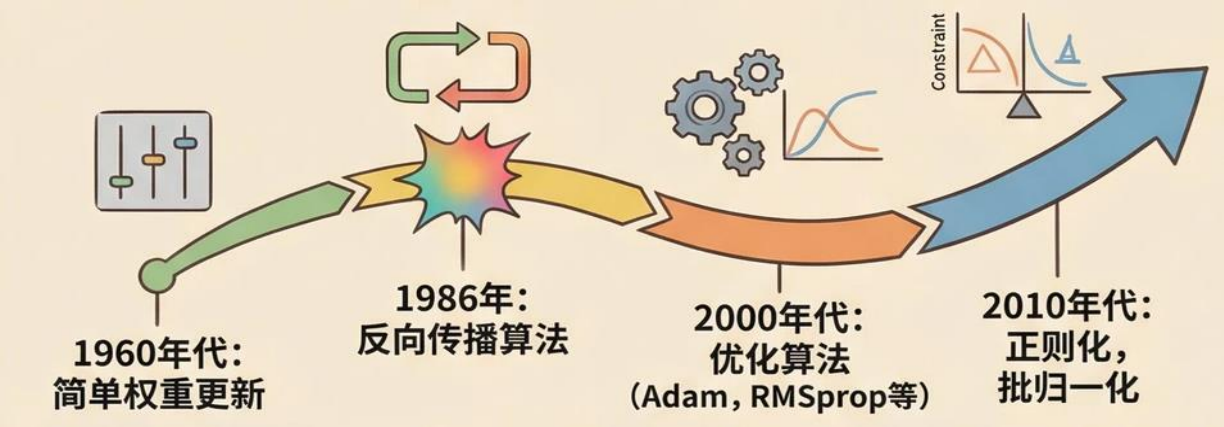
“极简公式”

将所有层合并，MLP的预测值可表示为（σ是第l层的激活函数）

$$\hat{y} = \sigma_L\left(W^{(L-1)}\sigma_{L-1}\left(\ldots\sigma_2\left(W^{(1)}x + b^{(1)}\right)\ldots + b^{(L-2)}\right) + b^{(L-1)}\right)$$

核心价值

MLP是**第一个能解决非线性问题的深度模型**，其“多层堆叠+非线性激活”的思想是所有深度模型(CNN、RNN、Transformer)的基础。理解MLP，就能理解深度学习的本质——**用分层的非线性变换，从数据中提取抽象特征。**



感谢您的观看

THANK YOU FOR WATCHING

汇报人:李欣怡